

Termo de Abertura do Projeto

- **Nome do Projeto:** Assistente de Voz para Automação Residencial
- **Data de Início:** 28/08/2026
- **Data de Término:** 04/12/2026

Visão Geral do Projeto

Descrição do Problema

Este projeto tem como objetivo proporcionar uma forma de interagir com uma automação residencial por comando de voz. Este produto é geralmente utilizado pelos mais diversos tipos usuários, mas com o objetivo de controlar, em exemplo: lâmpadas, ar condicionados e demais equipamentos inteligentes como tvs, geladeiras, caixas de som, projetores, etc. Sendo assim, o projeto busca integrar um Assistente de Voz ao próprio Home Assistant nativo do Raspberry Pi OS para automação residencial.

Estado da arte

Atualmente, existem soluções comerciais disponíveis no mercado como o Echo Dot (dispositivo com microfone e auto-falante) com a Alexa (assistente virtual que processa comandos de voz). Este é capaz de tocar música, participar de brincadeiras, acionar dispositivos inteligentes e ser controlado por voz e por aplicativo.

Há também no mercado a opção de utilizar ferramentas para processamento de áudio em texto rodando localmente na raspberry como o Vosk [1], o WHisper.cpp/FastWhisper [2], as quais provavelmente serão utilizadas.

Para o processamento de Palavra de Ativação (Wake Word) há bibliotecas como o Open Wake Word [3] que rodam localmente na Rasp e processam palavras como "Alexa!" ou "Ok, Google" para ativar o Whisper que irá processar a fala em texto. Há ainda o Speex [4] que serve para a supressão de ruído exclusivamente para a fala.

Para o processamento de linguagem e tradução para comandos, é possível tanto o uso de LLM local com o Ollama como também o uso de APIs gratuitas como o do Google Gemini que podem receber o texto e retornar um arquivo JSON formatado que é interpretado como comandos.

Por fim, para completar o ciclo de interação humana tornando o assistente capaz de responder audivelmente, utiliza-se a síntese de voz (Text-to-Speech - TTS). Entre as ferramentas modernas e otimizadas para processadores ARM, destaca-se o Piper TTS [6], um motor local e de código aberto que gera voz natural em Português do Brasil com baixíssima latência e consumo reduzido de memória RAM no Raspberry Pi 3.

Objetivos

Objetivo Geral

Desenvolver um sistema distribuído e embarcado de assistente de voz para automação residencial capaz de realizar processamento e detecção local de comandos de fala, interpretar a intenção do usuário por meio de

inteligência artificial generativa e executar ações físicas e lógicas em tempo real com atualização de interface gráfica.

Objetivos Específicos

- Integrar um motor de conversão de fala em texto (Speech-to-Text - STT) transcrição eficiente dos comandos de áudio capture após o acionamento da palavra de chave.
- Processar a fala e converter isto em comandos
- Gerenciar uma agenda, um calendario, uma lista de compras
- Comunicar-se com uma lâmpada inteligente
- Atender às necessidades dos stakeholders
- Implementar a síntese de voz local (Text-to-Speech - TTS) permitindo que o sistema responda audivelmente pela caixa de som conectada à placa.

Escopo do Projeto

O dispositivo deve ser capaz de:

- Receber comandos de voz
- Responder a comandos de voz de forma audível
- Buscar em um site informações de clima/tempo e falar como está.
- Responder qual dia é hoje.
- Ser capaz de agendar tarefas/eventos e armazená-los.
- Falar quais são os eventos/tarefas agendados para os próximos dias.
- Acionar uma lâmpada inteligente.
- Anotar itens em uma lista de compras.
- Falar a lista de compras quando solicitado.

Stakeholders

Possíveis usuários serão homens/mulheres que cuidam de uma casa com ou sem crianças que trabalham e precisam se lembrar de rotinas, eventos, controlar coisas em sua casa, listar coisas para comprar, saber como está o clima para se arrumar de manhã.

Recursos do Projeto

Membros da Equipe

Nome	Matrícula	Curso	Funções
André Jacinto Rodrigues	221007822	Engenharia Eletrônica	Líder

Orçamento estimado (R\$)

Item / Componente	Descrição / Modelo Sugerido	Qtd	Valor Estimado (R\$)
Raspberry Pi 3 (1GB)	Placa principal para rodar o backend, STT (Vosk) e controle	1	R\$ 0,00 (Já possui)
Fonte 5V 3A (USB-C)	Alimentação	1	R\$ 0,00 (Já possui)
Cartão MicroSD 32GB/64GB	Classe 10 / A1 para o SO e armazenamento dos modelos locais	1	R\$ 0,00 (Já possui)
Placa de Som USB Estéreo	Interface para entrada de microfone P2 e saída de áudio	1	R\$ 15,00 - R\$ 30,00
Microfone Omnidirecional / Webcam	Gravação de áudio	1	R\$ 0,00 (Já possui)
Caixa de Som Auxiliar	Caixa (Conexão via P2 na placa USB)	1	R\$ 0,00 (Já possui)
Custo de APIs (Gemini)	Plano gratuito (Free Tier) até 15 requisições/minuto	-	R\$ 0,00
Lâmpada Inteligente	Simples com conexão Wi-Fi	1	R\$ 20,00 - R\$ 30,00

Esforço estimado (horas)

A estimativa foi calculada considerando o desenvolvimento individual (1 integrante), dividindo a carga horária em **10 a 12 horas semanais** dedicadas ao projeto. A complexidade intermediária exige atenção especial à configuração de dependências do ecossistema C/Python e ao ajuste fino dos tempos de execução (*picos de CPU*) no Raspberry Pi 3.

Estimativa por Etapa do Projeto

Etapas de Desenvolvimento	Descrição das Atividades	Tempo Estimado (Semanas)	Horas Estimadas
1. Configuração do Hardware e SO	Instalação do Raspberry Pi OS Lite (64-bit), ajuste da placa de som USB, testes do microfone/caixa de som P2 e configuração do ambiente Python (venv).	1 semana	10h

Etapa de Desenvolvimento	Descrição das Atividades	Tempo Estimado (Semanas)	Horas Estimadas
2. Pipeline de Entrada (Wake Word)	Instalação do openwakeword , testes de escuta do áudio via <i>buffer</i> e ajuste da sensibilidade (<i>threshold</i>) para evitar falsos positivos.	1,5 semana	15h
3. Módulo de Transcrição (STT)	Integração do Vosk (modelo small-pt), criação do script de gravação pós-wake word e otimização da limpeza de memória RAM.	1,5 semana	15h
4. Processamento Semântico (Gemini API)	Configuração do SDK da Google Gemini API, definição das classes Pydantic (<i>Structured Outputs</i>) e tratamento de erros de conexão/JSON.	1 semana	10h
5. Módulo de Resposta Sonora (Piper TTS)	Compilação/instalação do Piper TTS , download do modelo em Português (pt_BR) e criação da função de reprodução de áudio síncrona.	1,5 semana	15h
6. Controle de Hardware (GPIO & Dispatcher)	Mapeamento dos pinos GPIO (relés), desenvolvimento do script principal (<i>loop/daemon</i>) unificando o fluxo sequencial completo.	1,5 semana	15h
7. Testes de Integração e Ajuste de Latência	Testes end-to-end de comandos de voz, calibração dos picos de processamento na CPU e ajustes no tempo de resposta do assistente.	1 semana	10h
8. Documentação Técnica e Validação Final	Redação do relatório final, organização do repositório de código e validação dos objetivos do projeto.	1 semana	10h

Resumo da Estimativa

- **Carga Horária Total: 90 horas de trabalho direto.**
- **Duração Estimada: 9 a 10 semanas** (aproximadamente 2,5 meses) de desenvolvimento contínuo individual.

Análise Sincera de Riscos e Desafios (Desenvolvimento Solo)

- **Compilação e Conflito de Dependências:** O maior risco de atraso na fase inicial é o processo de instalação de bibliotecas de áudio e C/C++ no Raspberry Pi OS (como o **PortAudio**, **sounddevice** e os *bindings* do Piper). Alocar 1 semana extra de margem para lidar com *builds* de ambiente é fundamental.

- **Gargalo de Processamento Unificado:** Por ser um desenvolvedor único, o projeto avança em modo estritamente sequencial. Caso um módulo (ex: o STT) demore mais do que o esperado para responder sem estourar a RAM do Pi 3, o progresso das etapas seguintes ficará pausado até a resolução do gargalo.

Referências

[1] D. Ward *et al.*, "openWakeWord: An open-source audio wake word detection framework," 2023. [Online]. Disponível em: <https://github.com/dscripka/openWakeWord>

[2] Xiph.Org Foundation, "The Speex speech codec and Speech Processing Library," 2023. [Online]. Disponível em: <https://www.speex.org/>

[3] Alpha Cephei, "Vosk Speech Recognition Toolkit," 2023. [Online]. Disponível em: <https://alphacephei.com/vosk/>

[4] G. Gerganov, "whisper.cpp: High-performance inference of OpenAI's Whisper model in C/C++," 2023. [Online]. Disponível em: <https://github.com/ggml-org/whisper.cpp>

[5] Google, "Gemini API Documentation: Structured Outputs with Pydantic," *Google AI for Developers*, 2024. [Online]. Disponível em: <https://ai.google.dev/>

[6] Rhasspy, "Piper: A fast, local neural text-to-speech system," 2023. [Online]. Disponível em: <https://github.com/rhasspy/piper>